講義ノート：ゼロ知識証明の主要方式

Groth16, BBS+/BBS, Bulletproofs, One-out-of-many proof

— 理論・構成・Toy 例・2026 年時点の応用地図 —

2026 年 3 月 18 日時点の標準化・実装状況を反映

概要

本稿は，ゼロ知識証明の文脈で頻繁に並べて語られる *Groth16*, *BBS+/BBS*, *Bulletproofs*, *One-out-of-many proof* を，講義用ノートとして一つにまとめたものである．ただし，この 4 方式は同じレイヤの競合技術ではない．Groth16 は汎用計算を短く証明する前処理型 zkSNARK，BBS+/BBS は選択的開示可能な複数メッセージ署名と匿名資格証明，Bulletproofs は trusted setup を使わない range proof / 回路証明，One-out-of-many proof は匿名集合所属を対数通信で示す Σ-protocol である．

本稿の目的は 2 つある．第 1 に，数式の意味を「何を圧縮しているのか」という視点で理解すること．第 2 に，2026 年時点で各方式が実際にどのような応用に使われているかを，**本番運用・標準化・研究** の 3 層に分けて整理することである．前半では理論と toy 例を説明し，後半では Zcash, Semaphore / World, Sui / IOTA, RISC Zero, W3C VC, OpenID4VP, Monero, Liquid / Elements, Firo, Triptych, Any-out-of-many などを軸に，「なぜその用途にその方式が選ばれるのか」を解説する．

目次

1 はじめに：4 方式は何を最適化しているのか 3

2 理論の前提：群・ペアリング・コミットメント 3

2.1 有限体・群・ペアリング 3

2.2 Pedersen commitment 4

2.3 Σ-protocol と Fiat–Shamir 4

3 Groth16 4

3.1 何を証明する方式か 4

3.2 QAP の形 5

3.3 Setup 5

3.4 Proof generation 6

3.5 Verification 6

3.6 Toy 例の最短版 6
3.7 Groth16 は近年どこで使われているか 7
3.8 Groth16 と応用の相性 10

4 BBS+/BBS 10
4.1 何をする方式か 10
4.2 古典的 BBS+ と現行 BBS 11
4.3 古典的 BBS+ の数学的心臓部 11
4.4 選択的開示 proof の骨格 11
4.5 現行 BBS 仕様の心臓部 12
4.6 Toy 例の最短版 12
4.7 BBS+/BBS は近年どこで使われているか 13
4.8 BBS が選ばれやすい応用条件 15

5 Bulletproofs 16
5.1 何を証明する方式か 16
5.2 Setup 16
5.3 range proof の核心 16
5.4 証明生成の骨格 17
5.5 Toy 例の最短版 17
5.6 Bulletproofs は近年どこで使われているか 18
5.7 Bulletproofs が選ばれやすい応用条件 20

6 One-out-of-many proof 20
6.1 何をする方式か 20
6.2 核心アイデア 20
6.3 証明生成と検証の最短版 21
6.4 Toy 例の最短版 22
6.5 One-out-of-many proof は近年どこで使われているか 22
6.6 One-out-of-many が選ばれやすい応用条件 25

7 **横断比較：4 方式の応用の違い** 25
7.1 まず一目で見る表 25
7.2 なぜ同じ「ゼロ知識」でも用途が違うのか 26

8 **2026 年時点の講義用まとめ** 26

1　はじめに：4 方式は何を最適化しているのか

最初に，本稿全体の地図を示す．この地図が見えていると，後の数式が「なぜその形なのか」がかなり分かりやすくなる．

- **Groth16** は，**任意のかなり広い計算**を非常に短い証明と軽い検証に圧縮する汎用 SNARK である．弱点は回路ごとの trusted setup だが，証明サイズと検証コストは抜群に小さい．そのため，**オンチェーン検証コストが厳しい場面**，**定数サイズ** *proof* **が欲しい場面**，**他方式のラップ用** *SNARK* で特に強い [1, 20, 21, 22, 26, 27].
- **BBS+/BBS** は，複数属性に対する定サイズ署名と，その署名に対する **選択的開示・** *unlinkable derived proof* のための方式である．したがって主戦場は，**デジタル資格証明・** *VC*・**デジタル身分証・アクセス資格** である [3, 4, 5, 6, 9, 10].
- **Bulletproofs** は，trusted setup なしで Pedersen commitment 上の *range proof* や比較的構造化された算術条件を示すのに向く．そのため，**秘密金額の妥当性証明**，*Confidential Transactions*，*proof of solvency / reserves* と相性がよい [12, 14, 29, 31].
- **One-out-of-many proof** は，**多数の候補のうち** *1* **つだけ正しい** ことを，どれかを隠したまま示す道具である．したがって **匿名集合所属**，*ring signature*，**匿名支払い**，**大きな** *anonymity set* **を使う資格証明・サインオン** と相性がよい [15, 16, 17, 18, 19].

■現況ラベル　以下では応用の現況を 3 つに分ける．

- [**本番運用**]　既に本番運用・本番プロトコル・公開実装で使われているもの
- [**標準化**]　標準仕様・相互運用仕様・制度設計に入っているもの
- [**研究・試験実装**]　有力な研究提案・試験実装・ロードマップ段階のもの

■事実と推測の区別　本稿ではできるだけ

「仕様・実装・運用として確認できる事実」

と

「その仕様から自然に導ける設計上の含意」

を区別する．特に「将来の主流になりそう」といった評価は推測であり，その場合はそう明記する．

2　理論の前提：群・ペアリング・コミットメント

2.1　有限体・群・ペアリング

- $\mathbb{F}$ は有限体，通常は $\mathbb{F}_p$ を想定する．

- $\mathbb{G}, G_1, G_2, G_T$ は位数 p の群である．
- ペアリングは
$$e : G_1 \times G_2 \to G_T$$
という双線形写像であり，
$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$$
を満たす．Groth16 と BBS+/BBS はこれを本質的に使う [1, 3, 5].

2.2 Pedersen commitment

Pedersen 型コミットメントは
$$\mathsf{Com}(m; r) = g^m h^r$$
と書ける．完全秘匿であり，離散対数仮定のもとで計算量的 binding である．Bulletproofs と One-out-of-many proof では，この種のコミットメントが中心になる [12, 15].

2.3 Σ-protocol と Fiat–Shamir

One-out-of-many proof や多くの Schnorr 型証明は
$$\text{commit} \to \text{challenge} \to \text{response}$$
という 3-move public-coin 形を持つ. Fiat–Shamir により challenge をハッシュから作れば, random oracle model で非対話化できる [12, 15].

3 Groth16

3.1 何を証明する方式か

Groth16 は，公開 statement ϕ と秘密 witness w が
$$(\phi, w) \in R$$
を満たすことを示す前処理型 zkSNARK である [1]. 有名なのは，証明が
$$\pi = (A, B, C)$$
という 3 個の群要素からなり，検証が少数の pairing で済む点である．

講義用に本質だけ先に言うと，Groth16 は

> **「回路制約全体を QAP という 1 本の多項式関係に変え，それを秘密評価点で圧縮し，pairing で一撃検証する」**

方式である．

3.2 QAP の形

変数を

$$a_0 = 1,\ a_1, \ldots, a_m$$

とし，$a_1, \ldots, a_\ell$ を公開入力，残りを秘密 witness とする．QAP では各変数に多項式

$$u_i(X),\ v_i(X),\ w_i(X)$$

を対応させ，制約全体を

$$\left(\sum_{i=0}^{m} a_i u_i(X)\right)\left(\sum_{i=0}^{m} a_i v_i(X)\right) = \left(\sum_{i=0}^{m} a_i w_i(X)\right) + h(X)t(X)$$

へ落とす [2, 1].

ここで

$$t(X) = \prod_{q=1}^{n}(X - r_q)$$

は制約点を根に持つ vanishing polynomial である．つまり「全制約点で 0」を「$t(X)$ で割り切れる」という 1 本の条件へ圧縮している．

3.3 Setup

Groth16 の setup は relation 固有であり，trapdoor

$$(\alpha, \beta, \gamma, \delta, s)$$

を選ぶ [1]. CRS には概念的に

- 秘密点 s の冪
$$G_1^{s^0}, \ldots, G_1^{s^{n-1}}, \qquad G_2^{s^0}, \ldots, G_2^{s^{n-1}}$$
- 公開入力用クエリ
$$K_i = G_1^{(\beta u_i(s) + \alpha v_i(s) + w_i(s))/\gamma}$$
- witness 用クエリ
$$L_i = G_1^{(\beta u_i(s) + \alpha v_i(s) + w_i(s))/\delta}$$
- quotient 用クエリ
$$Z_j = G_1^{s^j t(s)/\delta}$$

などが入る [1].

直感的には，prover は s 自体を知らなくても，$G_1^{s^k}$ が並んでいれば多項式 $f(X)$ に対して

$$G_1^{f(s)}$$

を指数の中で組み立てられる．

3.4 Proof generation

QAP の

$$U(X) = \sum_i a_i u_i(X), \quad V(X) = \sum_i a_i v_i(X), \quad W(X) = \sum_i a_i w_i(X)$$

を作り，

$$U(X)V(X) - W(X) = h(X)t(X)$$

となる $h(X)$ を得る．さらにランダム化

$$\rho, \eta \leftarrow \mathbb{F}$$

を用いて

$$\begin{aligned}
A &= G_1^{\alpha + U(s) + \rho\delta}, \\
B &= G_2^{\beta + V(s) + \eta\delta}, \\
C &= G_1^{\frac{\sum_{i=\ell+1}^{m} a_i(\beta u_i(s) + \alpha v_i(s) + w_i(s)) + h(s)t(s)}{\delta} + \eta(\alpha + U(s)) + \rho(\beta + V(s)) + \rho\eta\delta}.
\end{aligned}$$

と置く [1].

3.5 Verification

verifier は公開入力から

$$K_{\mathrm{pub}} = \prod_{i=0}^{\ell} K_i^{a_i}$$

（ただし $a_0 = 1$）を作り，

$$e(A, B) \stackrel{?}{=} e(G_1^{\alpha}, G_2^{\beta}) \cdot e(K_{\mathrm{pub}}, G_2^{\gamma}) \cdot e(C, G_2^{\delta})$$

をチェックする [1].

3.6 Toy 例の最短版

前回のノートを踏まえ，ここでは要点だけ再掲する．公開入力 $(x, z) = (10, 59)$，秘密 witness $y = 7$ に対し

$$z = y^2 + x$$

を証明する．中間変数 $m = y^2 = 49$ を置いて QAP を作ると

$$U(X) = 52X - 45, \qquad V(X) = 13 - 6X, \qquad W(X) = 10X + 39$$

で，

$$U(X)V(X) - W(X) = -312(X^2 - 3X + 2)$$

だから

$$h(X) = -312 \equiv 92 \pmod{101}.$$

trapdoor を

$$\alpha = 5,\ \beta = 7,\ \gamma = 11,\ \delta = 13,\ s = 3$$

とし，ランダム化を $\rho = \eta = 1$ とすると，証明は

$$A = G_1^{28}, \qquad B = G_2^{15}, \qquad C = G_1^{9}$$

となり，

$$e(A, B) = e(G_1, G_2)^{16}$$

と

$$e(G_1^{\alpha}, G_2^{\beta})e(K_{\mathrm{pub}}, G_2^{\gamma})e(C, G_2^{\delta}) = e(G_1, G_2)^{16}$$

が一致して受理される．この toy 例の価値は，*QAP* **の** *divisibility* **条件が，最終的に** *1* **本の** *pairing* **等式へ落ちる** ことを目で確認できる点にある．

3.7 Groth16 は近年どこで使われているか

3.7.1 総論

Groth16 の応用を理解する一番良い方法は，**「なぜ** *2020* **年代後半になってもまだ** *Groth16* **が使われるのか」**を考えることである．答えは比較的単純で，Groth16 は

1. 証明が非常に短い，
2. 検証が非常に安い，
3. BN254/BLS12-381 まわりの実装・ツール・検証コントラクトが非常に成熟している，

からである．したがって，

> **「prover は重くてもよいから，verifier を極限まで軽くしたい」**

場面で，今でも第一候補になりやすい．

3.7.2 [本番運用] Zcash の Sapling

現行 Zcash 仕様では，Sprout 以前の BCTV14 と対比して，**Sapling の Spend statement と Output statement は Groth16 と BLS12-381 で証明・検証される**．同じ仕様書は，NU5 以降の Orchard Action statement では Halo 2 が使われることも明記している [20].

この事実は重要である．なぜなら Zcash 自身が

> 「古い shielded pool の Sapling には Groth16 を残しつつ，新しい Orchard には Halo 2 を採用する」

という **世代交代の実例** だからである．

講義で強調すべき点は次の通りである．

- Groth16 は **2018–2026 の長い時間スパンで本番運用に耐えてきた.**
- ただし新規設計では universal / recursive friendly な方式へ移る圧力が強い.
- それでも既存資産・既存ウォレット・既存回路互換性のため，**Groth16 はすぐ消えない.**

これは，暗号方式の寿命が**「理論的により新しいものが出た」**だけでは終わらず，**「既存の実装と経済圏にどれだけ埋め込まれているか」**にも強く依存する好例である.

3.7.3 [本番運用] Ethereum 周辺の匿名集合・匿名シグナリング：Semaphore

Semaphore は，グループ membership を匿名に証明しつつ，投票・シグナル送信・rate-limited nullifier などを実現する Ethereum 向け基盤である．公式サイトは，ユーザが

> group membership を証明し，votes や endorsements などの signals を，元の identity を明かさずに送れる

と説明している [23].

さらに Semaphore のコントラクト資料は，`SemaphoreVerifier.sol` が

> **SnarkJS が生成する Groth16 verifier を拡張したもの**

であり，trusted setup 時に複数の回路深さに対する verification key を埋め込むと説明している [24].

ここから見えるのは，Groth16 の現代的な用途の一つが

> **オンチェーン匿名 membership / nullifier systems**

である，ということだ．この用途では，*proof* **を短くし，検証をできるだけ** *gas-efficient* **にしたい.** まさに Groth16 の得意分野である.

3.7.4 [本番運用] World / World ID における unique personhood

World の whitepaper は，World Network protocol が Semaphore をベースにしており，**ウォレットが Groth16 proof を生成して，オンチェーン identity set の公開鍵のどれか一つに対応する秘密鍵を知っていることと，アクションに対応する nullifier を計算したこと** を証明すると説明している [25].

同 whitepaper は応用として

- verified human であることの証明,
- WLD トークンの claim,
- proposal 上の投票,
- アプリごとの Sybil resistance

を挙げている [25].

さらに World は，登録時の *Merkle tree batch insertion* に対しても Groth16 proof を用いてい

る [25]．ここで学術的に面白いのは，

> **Groth16 が単なる membership proof 用途だけでなく，更新操作の正当性（バッチ挿入の before/after の整合性）証明にも使われている**

点である．

Groth16 の応用範囲は「ただの membership」より広く，**オンチェーンで** *cheap* **に検証したい状態遷移の正当性** 全般に及ぶ．

3.7.5 [本番運用] 汎用オンチェーン verifier API：Sui と IOTA

Sui の Move API は，

> NP-complete language で表せる任意の statement を，BN254 または BLS12-381 上の Groth16 で検証できる

と説明している [21]．IOTA の Move API も同様に，Circom で記述した statement を Groth16 で検証する例を載せている [22]．

この用途は，Zcash や Semaphore よりさらに一般的である．ここでは Groth16 は，特定アプリケーションの内部 primitive ではなく，

> **チェーンが提供する「一般-purpose verifier API」**

として扱われている．

講義でのポイントは，*Groth16* **はもはや単一プロトコル専用技術ではなく，「汎用証明をオンチェーンで受理するための標準部品」に近づいている面がある**ということだ．ただし trusted setup が circuit-specific であることは残るので，開発体験の単純さでは universal SRS 系より不利である．

3.7.6 [本番運用] / [研究・試験実装] zkVM の SNARK wrapper：RISC Zero, zkVerify など

RISC Zero の Ethereum 統合資料は，zkVM が生成する STARK proof を

> **Circom Groth16 SNARK に wrap すると，Ethereum 上でかなり小さく・速く検証できる**

と説明している [26]．

また zkVerify の説明は，多くの zkVM が STARK backend を使っているため，Ethereum 上での検証のために *Groth16 wrapper* を使うことが多いと明言している [27]．同時に zkVerify は，Groth16 verifier pallet を独立に持つ [28]．

ここでの応用は，もはや「Groth16 自身が元の計算を証明する」のではない．そうではなく，

> **別の巨大 proof system の最終出力を，EVM-friendly な小さな SNARK に変換する外装材**

として機能している．

これは 2020 年代後半の Groth16 を理解する上で重要である．Groth16 は最先端の proving backend ではなくなりつつあるが，**「最後の** *mile* **の検証コストを下げる** *wrapper***」**として非常に強い位置にいる．

3.8 Groth16 と応用の相性

Groth16 が選ばれやすい条件を講義的にまとめると，

1. verifier がチェーンや小型デバイスで重い処理をできない，
2. proof を calldata / bandwidth / storage の都合でできるだけ短くしたい，
3. relation-specific trusted setup を許容できる，
4. recursive friendliness よりも **単発** *proof* **の小ささ** を優先する，

の 4 つである．

このため，近年の典型用途は

- shielded payments（Zcash Sapling）
- anonymous set-membership / nullifier systems（Semaphore, World）
- 汎用オンチェーン verifier API（Sui, IOTA）
- zkVM / STARK system の wrapper proof（RISC Zero, verifier layers）

に集約される．

4 BBS+/BBS

4.1 何をする方式か

BBS+/BBS は，汎用計算証明ではなく，**複数メッセージ署名** と，その署名に対する **選択的開示付き** *proof of knowledge* を与える方式である [3, 4, 5, 6].

一番大事な直感は，

> **「署名付きデジタル証明書を，必要な項目だけ見せて提示できる」**

である．

運転免許証・学生証・学位証明・在籍証明・年齢属性・国籍属性のように，issuer が複数属性に署名した credential を，holder が後で

- 氏名は隠すが年齢帯だけ見せる
- 学校名は見せるが生年月日は隠す
- issuer の真正な署名があることだけは保証する

という使い方に向く [8, 6, 5].

4.2 古典的 BBS+ と現行 BBS

古典的な学術文献では，BBS+ 署名は

$$\sigma = (A, e, s)$$

として書かれることが多い [3]．一方，現行 IETF draft の BBS は Tessaro–Zhu 系の短い形を採り，本質的には

$$\sigma = (A, e)$$

へ整理されている [4, 5]．

実務的には，現在参照すべき仕様は

- IETF/IRTF の *BBS Signature Scheme*
- W3C の *Data Integrity BBS Cryptosuites v1.0*

である [5, 6]．

4.3 古典的 BBS+ の数学的心臓部

公開パラメータを

$$g_1 \in G_1, \quad g_2 \in G_2, \quad h_0, h_1, \ldots, h_L \in G_1$$

とし，issuer の秘密鍵を x，公開鍵を $w = g_2^x$ とする [3]．

メッセージ列 $(m_1, \ldots, m_L)$ に対する署名は

$$A = \left(g_1 h_0^s \prod_{i=1}^{L} h_i^{m_i} \right)^{1/(x+e)}, \qquad \sigma = (A, e, s)$$

で与えられる [3]．

検証は

$$e\big(A, w g_2^e\big) \stackrel{?}{=} e\left(g_1 h_0^s \prod_{i=1}^{L} h_i^{m_i}, g_2 \right)$$

である．

4.4 選択的開示 proof の骨格

holder は署名 $\sigma = (A, e, s)$ をランダム化し，

$$A_0 = A^{r_1}, \qquad \bar{A} = A_0^{-e} B^{r_1}, \qquad d = B^{r_1} h_0^{-r_2}, \qquad s_0 = s - r_2/r_1$$

などを作って，未開示メッセージと (e, r_2, r_3, s_0) の知識を Schnorr / Fiat–Shamir 型で証明する [3]．

重要なのは，開示集合 D と未開示集合 U を用いて

$$g_1 \prod_{i \in D} h_i^{m_i} = d^{r_3} h_0^{-s_0} \prod_{i \in U} h_i^{-m_i}$$

という形を作ることで，**開示属性だけが左辺に露出し，未開示属性が右辺の** *witness* **に吸収されることだ** [3].

4.5 現行 BBS 仕様の心臓部

IETF draft では，加法記法で

$$B = P_1 + Q_1 \cdot \mathsf{domain} + \sum_{i=1}^{L} H_i \cdot m_i, \qquad A = B \cdot (SK + e)^{-1}$$

と署名を作る [5]. proof 生成は署名をランダム化し，

$$D,\ \bar{A},\ \bar{B},\ T_1,\ T_2$$

などを作って challenge-response で未開示属性の知識を示す [5].

したがって BBS 系の本質は，

> **「署名そのものを proof object に変換し，disclosed messages だけを外に出し，rest を零知識 witness に押し込む」**

ことにある.

4.6 Toy 例の最短版

前回の数値例を短く再掲する．有限体 $\mathbb{F}_{101}$ の指数世界で

$$h_0 = g_1^5, \qquad h_1 = g_1^7, \qquad h_2 = g_1^{11}, \qquad x = 13, \qquad w = g_2^{13}$$

とし，メッセージを

$$m_1 = 20, \qquad m_2 = 9$$

とする.

$s = 17,\ e = 19$ を選ぶと

$$B = g_1 h_0^{17} h_1^{20} h_2^{9} = g_1^{22}, \qquad A = B^{1/(x+e)} = g_1^7.$$

よって署名は

$$\sigma = (A, e, s) = (g_1^7, 19, 17).$$

この署名をランダム化し，m_1 だけ開示して m_2 を隠すと，

$$\bar{A}/d = A_0^{-e} h_0^{r_2}, \qquad g_1 h_1^{m_1} = d^{r_3} h_0^{-s_0} h_2^{-m_2}$$

が成り立ち，Schnorr 型応答で未開示部分を隠したまま有効署名の知識を示せる．この toy 例の価値は，**署名済み属性ベクトルから，開示部分だけが明示的に見える** *relation* **を切り出せる** ことを可視化できる点にある．

4.7 BBS+/BBS は近年どこで使われているか

4.7.1 [標準化] Verifiable Credentials における selective disclosure

W3C VC Data Model 2.0 は，driver's license, education certificate, passport, healthcare data などを *Web* **上で暗号的に安全かつ** *privacy-respecting* **に扱う枠組み** を与える [8]．その上で W3C の BBS cryptosuite は，**BBS 署名を使って selective disclosure derived proof を生成する具体的アルゴリズム**を定義している [6]．

W3C BBS spec の holder-side algorithm は，base proof から *selectively disclosed details only* を含む derived proof を生成すると説明している [6]．つまり BBS の最重要応用は，

「VC を全部見せるのではなく，必要な claim だけ見せる」

ことにある．

授業で強調したいのは，BBS は「署名方式」だが，その価値は署名サイズや署名速度だけではなく，**証明に変換したときの** *privacy property* — unlinkable derived proof, disclosed subset の integrity, undisclosed subset の秘匿 — にある，という点だ．

4.7.2 [標準化] OpenID4VP / DCQL と組み合わさる提示プロトコル

OpenID for Verifiable Presentations 1.0 は，verifier が DCQL で必要 claim を要求し，wallet が credential を提示するプロトコルである [9]．同仕様は example の `cryptosuite_values` に

`"bbs-2023"`

を含めており，BBS ベースの VC presentation が**アプリケーション・プロトコル層の相互運用枠組み** に入っていることが分かる [9]．

これは重要である．BBS は単体では

$$\text{署名} + \text{選択的開示 proof}$$

に過ぎないが，OpenID4VP のようなプロトコルに入ることで初めて

$$\text{wallet} \leftrightarrow \text{verifier}$$

の実システムで使えるようになる．暗号方式として優れていても，*presentation request / consent / claim query / replay protection* の層がないと，現実の credential system にはならない．

4.7.3 [標準化] unlinkability だけでなく，holder binding / pseudonym へ

W3C BBS cryptosuite は，単なる selective disclosure だけではなく，

- **Anonymous Holder Binding**
- **Credential-Bound Pseudonyms**

という機能も扱っている [6].

これは応用の幅を大きく広げる．たとえば credential を

> 見せるたびに全く unlinkable であること

が望ましい場面もあれば,

> 同一 verifier に対しては疑似名で継続利用したい

場面もある．アクセス制御，サブスクリプション，大学内の継続資格確認，匿名だが継続課金可能なサービスなどでは，後者が必要になる．

4.7.4 **[標準化]** / **[研究・試験実装]** EUDI / eIDAS2 と selective disclosure

ETSI の 2025 年 feasibility study は，eIDAS2 の文脈で selective disclosure が *European Digital Identity Wallet* **の基本設計機能**であることを述べ，さらに BBS+/BBS# の eIDAS2 への適用可能性を独立章で検討している [10].

ここでは慎重な書き方が必要である．[**注意**] 2026 年 3 月時点で,

> 「EUDI wallet が BBS を本番採用した」

と断定するのは強すぎる．仕様・政策・実装系はなお流動的である．しかし少なくとも,

- selective disclosure が制度的に強く要請されていること,
- ETSI が BBS 系を候補技術として精査していること,
- W3C / IETF 側でも BBS の標準化が進んでいること,

は事実である [10, 7, 5, 6].

したがって BBS は,

> **匿名資格証明の「実務標準候補」**

としての地位をかなり強めていると言える．

4.7.5 **[研究・試験実装]** per-verifier linkability

IETF の *BBS per Verifier Linkability* draft は，元来 unlinkable である BBS presentation に対して,

> 監視・異常検知・pseudonymous identity・monetization

などのために，*verifier* **限定で** *linkability* **を持たせる** 応用を扱っている [11].

この拡張が示しているのは，BBS の応用が単なる「最低限だけ見せる credential」から,

プライバシーと継続的サービス提供のバランスを取る credential

へ広がっているということだ．現代の credential system では，unlinkability は常に絶対善ではない．異常利用検知や abuse mitigation を考えると，*linkability* **をゼロにするか，一定範囲で残すか**を設計選択として扱う必要がある．BBS 系はこの設計空間をうまく切り分けられる．

4.7.6 [研究・試験実装] eIDAS 互換の匿名資格証明

2025 年の *Making BBS Anonymous Credentials eIDAS 2.0 Compliant* は，BBS 系匿名資格証明を secure hardware / certification 文脈でどう適合させるかを研究している [36]．この方向は，BBS が「学術的には美しいが実装認証が難しい方式」から，

認証制度や secure element まで含めた現実の ID wallet 技術

へ踏み込んでいることを示す．

4.8 BBS が選ばれやすい応用条件

BBS+/BBS が自然な選択になるのは，おおむね次の条件が揃うときである．

1. statement が「有効署名付き属性列の一部提示」である，
2. 任意計算全般を証明したいわけではない，
3. unlinkable presentation を重視する，
4. issuer / holder / verifier の 3 者モデルが明確である，
5. attribute-level data minimization が政策・制度上重要である．

したがって BBS の主戦場は，

- 運転免許証・学生証・学位・在籍証明・会員資格証明
- 年齢確認・居住地確認・所属確認
- pseudonymous access credentials
- wallet-based VC presentation

である．

一方，

「年齢が 18 以上」や「スコアが閾値以上」

のような述語を **ネイティブに** 証明するわけではない．その場合は range proof や追加 ZK と組み合わせる必要がある．ここで Bulletproofs や一般 zkSNARK との役割分担が生まれる．

5 Bulletproofs

5.1 何を証明する方式か

Bulletproofs は，Pedersen commitment された秘密値に関する range proof や，より一般の算術回路証明を，*trusted setup* **なし**で与える方式である [12].

典型的な statement は

$$V = g^v h^\gamma$$

に対して

$$0 \leq v < 2^n$$

を，v を明かさずに示すことである [12].

講義的に一言で言うと，Bulletproofs は

> **「bit 分解・ランダム線形結合・1 点評価・inner-product folding により，多数の制約を対数通信へ潰す方式」**

である．

5.2 Setup

公開パラメータとして

$$g, h, g_1, \ldots, g_n, h_1, \ldots, h_n, u$$

を持つ [12]. Groth16 と違って relation-specific な CRS はない．公開 generator 群を共有すればよい．この意味で Bulletproofs は transparent setup である [12].

5.3 range proof の核心

秘密値 v を bit vector

$$\boldsymbol{a}_L \in \{0,1\}^n$$

に分解して

$$\langle \boldsymbol{a}_L, \boldsymbol{2}^n \rangle = v$$

とする．さらに

$$\boldsymbol{a}_R = \boldsymbol{a}_L - \boldsymbol{1}^n$$

とおけば

$$\boldsymbol{a}_L \circ \boldsymbol{a}_R = \boldsymbol{0}^n$$

が bitness 条件になる [12].

Bulletproofs の発想は，これらの多数の条件を challenge y, z によって

$$\left\langle \boldsymbol{a}_L - z\mathbf{1}^n,\, \boldsymbol{y}^n \circ (\boldsymbol{a}_R + z\mathbf{1}^n) + z^2\mathbf{2}^n, = \right\rangle z^2 v + \delta(y, z)$$

という 1 本の inner-product condition にまとめることにある [12].

5.4 証明生成の骨格

1. $\boldsymbol{a}_L, \boldsymbol{a}_R$ をベクトルコミットメント A に隠す.
2. blinding vectors $\boldsymbol{s}_L, \boldsymbol{s}_R$ をコミットした S を送る.
3. verifier から y, z を受け取り，

$$l(X),\ r(X),\ t(X) = \langle l(X), r(X) \rangle$$

   を定義する.
4. t_1, t_2 を T_1, T_2 へコミットする.
5. verifier から x を受け取り，

$$\boldsymbol{l} = l(x),\ \boldsymbol{r} = r(x),\ \hat{t} = \langle \boldsymbol{l}, \boldsymbol{r} \rangle$$

   を作る.
6. 最後に inner-product argument で $\boldsymbol{l}, \boldsymbol{r}$ の送信を $\log n$ に圧縮する.

[12]

5.5 Toy 例の最短版

4-bit 範囲で

$$v = 11, \qquad \gamma = 13, \qquad V = g^{11} h^{13}$$

とする．bit 分解は

$$\boldsymbol{a}_L = (1, 1, 0, 1), \qquad \boldsymbol{a}_R = (0, 0, -1, 0).$$

challenge を

$$y = 3, \qquad z = 2$$

と取ると

$$\boldsymbol{a}_L - z\mathbf{1}^4 = (-1, -1, -2, -1),$$
$$\boldsymbol{y}^4 \circ (\boldsymbol{a}_R + z\mathbf{1}^4) + z^2\mathbf{2}^4 = (6, 14, 25, 86),$$

内積は

$$46 \pmod{101}$$

になる.

他方

$$z^2 v + \delta(y, z) = 4 \cdot 11 + 2 = 46.$$

したがって多数の制約が 1 本の内積へまとめられている [12].

5.6 Bulletproofs は近年どこで使われているか

5.6.1 総論

Bulletproofs の応用は，Groth16 や BBS より分かりやすい．基本的に

> **「秘密金額・秘密数量・秘密残高が，ちゃんと正の範囲に入っていることを setup なしで示したい」**

場面で強い．

つまり Bulletproofs の現代的アイデンティティは，*general-purpose universal proof system* というよりも，

> **「range proof の第一級市民」**

である．

5.6.2 [本番運用] Monero の Bulletproofs+

Monero は 2022 年 8 月の network upgrade で **Bulletproofs+** を導入し，公式発表は

> typical transaction size を約 5–7% 削減し，typical verification performance も約 5–7% 改善する

と説明している [29].

さらに Monero の 2020 年公式記事は，Bulletproofs+ が

- 通常の 2-output proof で 96 bytes 小さい
- proof generation が約 10.2% 高速
- batch verification が最大で約 10.8% 高速

であると述べている [30].

ここから見える Bulletproofs の強みは明確である．Monero の Confidential Transaction では金額が hidden なので，各 output が正しい範囲にあることを示さなければ，オーバーフローや負値を悪用した inflation が起こりうる．つまり Bulletproofs 系は

> **「秘密金額を持つ支払い系が壊れないための range proof」**

として使われている [30].

これは BBS や Groth16 では代替しにくい Bulletproofs の典型用途である．

5.6.3 [本番運用] / [研究・試験実装] Confidential Transactions と multi-asset privacy

Stanford の Bulletproofs 解説ページは，Bulletproofs が *Bitcoin and other cryptocurrencies* の *efficient confidential transactions* のために設計されたことを明記している [14]．さらに Liquid

whitepaper は，Confidential Transactions と Bulletproofs の文脈で，

- 金額を Pedersen commitment に隠すこと
- proof-of-reserve system に使えること
- multi-asset transaction によって atomic exchange を private にできること

を説明している [31].

ただし，ここは厳密さが必要である．[**注意**] Blockstream の 2023 年記事は，**現在の Liquid CT は Borromean range proofs を使っている** と書いており，その上で Bulletproofs++ を *Liquid* **へ導入する次の大きな改善候補** として説明している [32].

同記事は BP++ について

- 64-bit range proof size が 416 bytes
- これは BP より 39% 小さい
- 現行 Liquid range proof より約 10 倍小さい
- proving / verification が BP より約 4 倍高速

と述べている [32].

この状況は教育上とても面白い．Bulletproofs は

本番運用（Monero）と，次世代 upgrade 候補（Liquid / Elements）

の両方で重要である．つまり「すでに使われている primitive」であると同時に，「まだ十分大きな改良余地がある primitive」でもある．

5.6.4 [研究・試験実装] solvency proofs, shuffles, confidential smart contracts

Stanford の Bulletproofs 解説は，本方式の用途として

- proofs of solvency
- short verifiable shuffles
- confidential smart contracts
- Sigma-protocol の drop-in replacement

を挙げている [14].

これは，Bulletproofs の価値が「送金金額の range proof」だけではないことを示す．Pedersen commitment と inner-product argument の組み合わせは，

「線形代数的な制約を，trusted setup なしでそこそこ短く示したい」

場面で幅広く使える．

ただし，ここでも誇張は避けるべきである．[**注意**] 実運用の厚みという意味では，2026 年時点でも Bulletproofs の本番用途の中心はやはり

Confidential Transactions とその周辺

である．他の用途は依然として研究・設計空間の色が強い．

5.7 Bulletproofs が選ばれやすい応用条件

Bulletproofs が自然に選ばれるのは，おおむね次の条件が揃うときである．

1. trusted setup を避けたい，
2. statement が committed values の range / linear relation に近い，
3. verifier は多少重くてもよい，
4. proof を定数サイズにはできなくても対数サイズなら許容できる，
5. pairing ベースより離散対数系の透明性を重視する．

したがって，Bulletproofs の代表用途は

- hidden amount の正当性証明
- confidential transactions
- multi-asset confidential accounting
- solvency / reserve / balance 系の秘匿監査

である．

6 One-out-of-many proof

6.1 何をする方式か

One-out-of-many proof は，公開コミットメント列

$$c_0, c_1, \ldots, c_{N-1}$$

に対して

$$\exists \ell, r: \ c_\ell = \mathsf{Com}(0; r)$$

を，どの ℓ かを隠したまま証明する方式である [15].

直感的には

$$c_0 \text{ が正しい } \vee c_1 \text{ が正しい } \vee \cdots \vee c_{N-1} \text{ が正しい}$$

という巨大 OR 証明の圧縮版である．

6.2 核心アイデア

$N = 2^n$ として秘密添字 ℓ を

$$\ell = \ell_1 \ell_2 \cdots \ell_n, \qquad \ell_j \in \{0, 1\}$$

に分解する [15]. 各 bit に対して

$$f_j = \ell_j x + a_j$$

を定義し，

$$f_{j,1}(x) = f_j, \qquad f_{j,0}(x) = x - f_j$$

と置く．

各公開添字 $i = i_1 \cdots i_n$ に対して

$$p_i(x) = \prod_{j=1}^{n} f_{j,i_j}(x)$$

を作ると

$$p_i(x) = \delta_{i,\ell} x^n + \sum_{k=0}^{n-1} p_{i,k} x^k$$

になる [15]．つまり

- $i = \ell$ のときだけ x^n 項が立つ
- それ以外は高々 $n-1$ 次

である．

低次項を補助コミットメント

$$D_k = \prod_i c_i^{p_{i,k}} \mathsf{Com}(0; \rho_k)$$

で打ち消すと，最後に秘密添字の項だけが残る [15]．

6.3 証明生成と検証の最短版

prover は各 bit について

$$C_{\ell,j} = \mathsf{Com}(\ell_j; r_j), \quad C_{a,j} = \mathsf{Com}(a_j; s_j), \quad C_{b,j} = \mathsf{Com}(\ell_j a_j; t_j)$$

を送り，各 k について D_k を作る [15]．

challenge x のあとで

$$f_j = \ell_j x + a_j, \qquad z_{a,j} = r_j x + s_j, \qquad z_{b,j} = r_j(x - f_j) + t_j,$$

および

$$z_d = r x^n - \sum_{k=0}^{n-1} \rho_k x^k$$

を返す [15]．

verifier は

$$C_{\ell,j}^{x} C_{a,j} \stackrel{?}{=} \mathsf{Com}(f_j; z_{a,j}), \qquad C_{\ell,j}^{x-f_j} C_{b,j} \stackrel{?}{=} \mathsf{Com}(0; z_{b,j})$$

で各 bit の正しさを確認し，最後に

$$\prod_{i=0}^{N-1} c_i^{\prod_j f_{j,i_j}} \cdot \prod_{k=0}^{n-1} D_k^{-x^k} \stackrel{?}{=} \mathsf{Com}(0; z_d)$$

を確認する [15]．

6.4 Toy 例の最短版

$N = 4$ で

$$c_0 = \mathsf{Com}(4; 11), \quad c_1 = \mathsf{Com}(7; 13), \quad c_2 = \mathsf{Com}(0; 17), \quad c_3 = \mathsf{Com}(9; 19)$$

とする．秘密添字は

$$\ell = 2 = (10)_2$$

である．bit に対して $a_1 = 5, a_2 = 8$ を選ぶと

$$p_{00}(x) = 40 - 5x, \quad p_{01}(x) = -40, \quad p_{10}(x) = x^2 - 3x - 40, \quad p_{11}(x) = 8x + 40.$$

このうち $p_{10}(x)$ **だけが** x^2 **項を持つ**．この性質が one-out-of-many proof の全てである．

6.5 One-out-of-many proof は近年どこで使われているか

6.5.1 総論

One-out-of-many proof の応用は，実はかなり広いが，どれも

「公開集合の中の 1 要素に対応する秘密を持つ」

という同じ骨格を持つ．

このため主な応用先は

- 匿名支払い
- linkable ring signature
- anonymous set-membership
- anonymity-set based sign-on / credentials

である．

6.5.2 [本番運用] / [研究・試験実装] Firo の Sigma, Lelantus, Lelantus Spark

Firo の研究ページは，**One-out-of-many proofs が Sigma の基礎であり，Sigma は Zerocoin から trusted setup を除去して proof を縮小した** と説明する [33]．同じページは，**Sigma は Lelantus に置き換わったが，基礎にある OOOMPs は Lelantus と Lelantus Spark に今も使われている**とも述べている [33]．

さらに Firo のトップページは，**Lelantus Spark が trustless, on-chain privacy と高い anonymity set を提供し，Spark Assets によって privacy-preserving tokens, voting, private tokenized assets にも拡張される**と説明している [34]．

ここで重要なのは，one-out-of-many proof そのものが単独でエンドユーザ向けに見えるわけではないという点である．実際にユーザが触れるのは Lelantus や Spark のような payment protocol であり，one-out-of-many はその内部で

「この coin / mint / note のどれか一つに正当に対応している」

ことを示す subroutine として働く．

6.5.3 [研究・試験実装] Monero Research Lab：Triptych と Arcturus

Monero Research Lab の研究紹介は，Triptych を

commitment openings to zero の知識証明の一般化に基づく，trusted setup を要しない logarithmic-sized linkable ring signatures

として説明し，confidential transaction model への応用を明示している [35, 16].

さらに同ページは Arcturus を，

単一 proof の中で複数 commitments to zero，VRF linking tag，value balance を同時に扱う

方式として紹介している [35].

教育上ここで重要なのは，Triptych / Arcturus を**「別方式」**と見るだけでなく，

one-out-of-many / opening-to-zero 型発想の現代的発展

として見ることである．そうすると，なぜ匿名集合のサイズと proof サイズの関係が対数になるのか，なぜ linkability が自然に入るのか，なぜ confidential transactions へ伸びるのか，が見えやすい．

6.5.4 [研究・試験実装] HOOOMP：階層化による高速化

HOOOMP (*Hierarchical One-out-of-Many Proofs With Applications to Blockchain Privacy and Ring Signatures*) は，one-out-of-many proofs の proving time を一桁改善する新しい構成として提案され，論文自身が

- Anonymous Zether
- Zerocoin
- Lelantus
- Triptych, Triptych-2

を building block の応用先として挙げている [17].

この論文のメッセージは明快である．one-out-of-many proof は通信が対数でも，prover / verifier 計算は依然として集合サイズの影響を強く受ける．だから近年の主要課題は

匿名集合を大きくしたまま，proving / verification をどこまで実用速度へ落とせるか

にある．

6.5.5 [研究・試験実装] Any-out-of-many：複数秘密への拡張

Any-out-of-many proof は，one-out-of-many を

1 secret out of many

から

arbitrarily many secrets out of many

へ拡張し，RingCT protocol へ応用する [18]．この方向は，匿名支払いで複数 input を扱うときに本質的である．

one-out-of-many の原型は

「正当な coin が 1 つある」

に強いが，実際の payment system では

「複数 input を同時に消費する」

場面が普通にある．すると匿名性・通信量・計算量・balance proof をどう同時に満たすかが難しくなる．Any-out-of-many はこの gap を埋める試みとして重要である．

6.5.6 [研究・試験実装] identity / sign-on 系：Anonymous Self-Credentials

2025 年の *Anonymous Self-Credentials and their Application to Single-Sign-On* は，anonymity set 上の master identities に対して，ユーザが *one-out-of-many proofs, accumulators, ring signatures* などを用いて membership を自己証明する sign-on の文脈を整理している [19]．

この論文が示すのは，one-out-of-many proof がもはや *privacy coin* **専用** ではないことだ．集合所属を匿名に示すという能力は，

- single-sign-on
- privacy-preserving sign-up
- Sybil-resistant pseudonyms
- anonymity-set based credentials

へも自然に広がる [19]．

つまり one-out-of-many proof の本質は

coin を spend すること

ではなく，

公開匿名集合のどれか一つに属することを，どれかを隠したまま示すこと

であり，この抽象化を理解すると応用範囲が一気に広く見える．

6.6 One-out-of-many が選ばれやすい応用条件

この方式が自然に選ばれるのは，次の条件が揃うときである．

1. statement が「集合所属」に近い，
2. secret index を隠したい，
3. anonymity set を大きく取りたい，
4. proof size を線形ではなく対数にしたい，
5. trusted setup を避けたい．

したがって代表用途は

- privacy coins の spend membership
- linkable ring signatures
- anonymous set-membership credentials
- anonymity-set based sign-on / pseudonym systems

である．

7 横断比較：4 方式の応用の違い

7.1 まず一目で見る表

方式	何を最適化するか	典型 statement	近年の代表用途
Groth16	汎用計算の極小 proof と軽い検証	任意回路 / QAP relation	Zcash Sapling, Semaphore / World, Move API によるオンチェーン verifier, zkVM / STARK wrapper
BBS+/BBS	署名済み属性列の選択的開示	有効署名付き多属性 credential	VC, デジタル資格証明, 匿名提示, OpenID4VP presentation, holder binding / pseudonym
Bulletproofs	transparent な range / linear proof	$V = g^v h^\gamma$, $0 \leq v < 2^n$ など	Monero, confidential transactions, proof of reserves, multi-asset confidential accounting, BP++ roadmap

方式	何を最適化するか	典型 statement	近年の代表用途
One-out-of-many	匿名集合所属の対数通信	$\exists \ell : c_\ell = \mathsf{Com}(0; r)$	Sigma / Lelantus / Spark, Triptych / Arcturus, Any-out-of-many RingCT, anonymity-set sign-on

7.2 なぜ同じ「ゼロ知識」でも用途が違うのか

■Groth16　Groth16 は **計算そのもの** を証明する．だから membership, hashing, Merkle updates, arbitrary predicates を一つの回路に詰め込める．ただし setup が重い．

■BBS+/BBS　BBS は **署名済み属性の最小開示** が主目的である．「本物の credential を持っているか」という statement に対して最短距離で解を出す．逆に「複雑な arbitrary arithmetic relation」は得意ではない．

■Bulletproofs　Bulletproofs は *Pedersen commitments* **上の** *range / linear algebra* に強い．だから hidden amount, hidden balance, hidden quantity の証明に非常に自然である．

■One-out-of-many　One-out-of-many は **秘密添字を持つ集合所属** に強い．だから coin ownership, ring membership, anonymity-set sign-on へ自然に伸びる．

8 2026 年時点の講義用まとめ

最後に，講義で学生に最も伝えたいポイントを 4 行でまとめる．

1. **Groth16** は「何でも証明できる」よりも，**「小さく・安く検証できる」**が現代的価値であり，とりわけオンチェーン検証・wrapper SNARK・匿名 membership 系で強い [20, 24, 25, 26, 21, 22].
2. **BBS+/BBS** は「資格証明を最小限だけ見せる」ための道具であり，W3C / IETF / OpenID4VP / EUDI 文脈で意味を持つ．主戦場は **デジタル資格証明・アイデンティティ** である [5, 6, 9, 10, 8].
3. **Bulletproofs** は「trusted setup なしで hidden values を扱う」ための道具であり，典型用途は **秘密金額・秘密残高の range proof** である．Monero は本番例，Liquid / Elements は今も活発な適用先である [29, 30, 14, 32, 31].
4. **One-out-of-many proof** は「大勢の中の 1 人／1 coin であることを匿名に示す」ための道具であり，privacy coins だけでなく，**ring signatures, anonymity sets, sign-on / credentials** にも伸びている [15, 33, 35, 17, 18, 19].

■総括　この 4 方式は「ゼロ知識証明」という一点でつながってはいるが，設計思想はかなり違う．学生にとって重要なのは，

「どの方式がどの statement に対して自然か」

を見抜くことである．暗号方式の比較は proof size や proving time の数字だけでは不十分であり，

対象 statement，運用層，標準化層，制度層まで含めて評価しなければならない

という点が，2020 年代後半の ZK エコシステムの最重要 lesson である．

参考文献

[1] Jens Groth, On the Size of Pairing-Based Non-interactive Arguments. In *EUROCRYPT 2016*, 2016.

[2] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova, Quadratic Span Programs and Succinct NIZKs without PCPs. In *EUROCRYPT 2013*, 2013.

[3] Jan Camenisch, Manu Drijvers, and Anja Lehmann, Anonymous Attestation Using the Strong Diffie Hellman Assumption Revisited. *IACR ePrint 2016/663*, 2016.

[4] Stefano Tessaro and Chenzhi Zhu, Revisiting BBS Signatures. In *EUROCRYPT 2023*, 2023.

[5] Tobias Looker, Vasilis Kalos, Andrew Whitehead, Mike Lodder, et al., The BBS Signature Scheme. IRTF CFRG Internet-Draft, draft-irtf-cfrg-bbs-signatures-10, 2026. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/

[6] W3C, Data Integrity BBS Cryptosuites v1.0. Candidate Recommendation Draft. https://www.w3.org/TR/vc-di-bbs/

[7] W3C, Verifiable Credentials Overview. 2025. https://www.w3.org/TR/vc-overview/

[8] W3C, Verifiable Credentials Data Model v2.0. Recommendation, 2025. https://www.w3.org/TR/vc-data-model-2.0/

[9] OpenID Foundation, OpenID for Verifiable Presentations 1.0. 2025. https://openid.net/specs/openid-4-verifiable-presentations-1_0.html

[10] ETSI, TR 119 476-1 V1.3.1: Selective disclosure and zero-knowledge proofs applied to Electronic Attestation of Attributes; Part 1: Feasibility study. 2025. https://www.etsi.org/deliver/etsi_tr/119400_119499/11947601/01.03.01_60/tr_11947601v010301p.pdf

[11] IRTF CFRG Internet-Draft, BBS per Verifier Linkability. 2025. https://www.ietf.org/archive/id/draft-irtf-cfrg-bbs-per-verifier-linkability-02.html

[12] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell, Bulletproofs: Short Proofs for Confidential Transactions and More. In *IEEE*

Symposium on Security and Privacy, 2018.

[13] Heewon Chung, Kyoohyung Han, Chanyang Ju, Myungsun Kim, and Jae Hong Seo, Bulletproofs+. *IACR ePrint 2020/735*, 2020.

[14] Stanford Applied Crypto Group, Bulletproofs: Short Proofs for Confidential Transactions and More. https://crypto.stanford.edu/bulletproofs/

[15] Jens Groth and Markulf Kohlweiss, One-Out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin. In *EUROCRYPT 2015*, 2015.

[16] Sarang Noether, Brandon Goodell, et al., Triptych: Logarithmic-Sized Linkable Ring Signatures with Applications. *IACR ePrint 2020/018*, 2020.

[17] Aram Jivanyan and Tigran Mamikonyan, Hierarchical One-out-of-Many Proofs With Applications to Blockchain Privacy and Ring Signatures. *IACR ePrint 2020/430*, 2020.

[18] Tianyu Zheng, Shang Gao, Yubo Song, and Bin Xiao, Leaking Arbitrarily Many Secrets: Any-out-of-Many Proofs and Applications to RingCT Protocols. In *IEEE Symposium on Security and Privacy*, 2023.

[19] Jayamine Alupotha, Mariarosaria Barbaraci, Ioannis Kaklamanis, Abhimanyu Rawat, Christian Cachin, and Fan Zhang, Anonymous Self-Credentials and their Application to Single-Sign-On. *IACR ePrint 2025/618*, 2025.

[20] Zcash Foundation / ECC, Zcash Protocol Specification, Version 2025.6.3 [NU6.1]. 2025. https://zips.z.cash/protocol/protocol.pdf

[21] Sui Documentation, Groth16. https://docs.sui.io/guides/developer/cryptography/groth16

[22] IOTA Documentation, Groth16. https://docs.iota.org/developer/cryptography/on-chain/groth16

[23] Semaphore, Semaphore Home / Project Showcase. https://semaphore.pse.dev/

[24] Semaphore Documentation, Contracts. https://docs.semaphore.pse.dev/technical-reference/contracts

[25] World, World Whitepaper. https://whitepaper.world.org/

[26] RISC Zero Developer Docs, RISC Zero on Ethereum. https://dev.risczero.com/api/blockchain-integration/risc-zero-on-eth

[27] zkVerify Documentation, What is zkVerify. https://docs.zkverify.io/

[28] zkVerify Documentation, Groth16 Verifier. https://docs.zkverify.io/architecture/verification_pallets/groth16

[29] Monero, Monero will undergo a network upgrade on 13th August, 2022. 2022. https://www.getmonero.org/2022/04/20/network-upgrade-july-2022.html

[30] Monero, Bulletproofs+ in Monero. 2020. https://www.getmonero.org/2020/12/24/Bulletproofs%2B-in-Monero.html

[31] Jonas Nick et al., Liquid: A Bitcoin Sidechain. 2020. https://blockstream.com/assets/

downloads/pdf/liquid-whitepaper.pdf

[32] Blockstream, Bulletproofs++: A Step Towards Fully Unlinkable Transactions With Multiple Asset Types. 2023. https://blog.blockstream.com/bulletproofs-a-step-towards-fully-anonymous-transactions-with-multiple-asset-types/

[33] Firo, Research. https://firo.org/about/research/

[34] Firo, Home. https://firo.org/

[35] Monero Research Lab, Monero Research Lab. https://www.getmonero.org/resources/research-lab/

[36] Nicolas Desmoulins, Antoine Dumanois, Seyni Kane, and Jacques Traoré, Making BBS Anonymous Credentials eIDAS 2.0 Compliant. *IACR ePrint 2025/619*, 2025.